
Employee Performance and Productivity Analysis

A SQL & Power BI Data Analytics Project

Dataset Size: 100,000 Employee Records
Database: employee_db
Table Name: employee
Total Columns: 20
Data Source: Kaggle

Tools Used

MySQL Workbench 8.0 • Microsoft Power BI • Microsoft Excel

Prepared By
Roshan Kumar

June 30, 2026

Table of Contents

1	Project Overview	2
2	Project Objectives	2
3	Dataset Information	2
4	Database Schema	2
5	Data Inspection	3
6	Data Quality Check	3
7	Exploratory Data Analysis (EDA)	4
7.1	Section A — Employee Overview	4
7.2	Section B — Salary Analysis	5
7.3	Section C — Performance Analysis	6
7.4	Section D — Overtime & Productivity Analysis	6
7.5	Section E — Satisfaction Analysis	7
7.6	Section F — Promotion Analysis	8
7.7	Section G — Experience Analysis	9
7.8	Section H — Hiring Trend (Date Functions)	10
8	Advanced SQL	10
8.1	Section I — CTE (Common Table Expression)	10
8.2	Section J — Window Functions	11
9	Power BI Dashboard	14
10	Business Insights	15
11	Conclusion	15

1. Project Overview

This project focuses on analyzing employee performance and productivity using SQL and Power BI. The dataset contains employee demographic information, salary, work experience, overtime, promotions, performance scores, and employee satisfaction.

SQL is used to perform data inspection, data quality checks, exploratory data analysis (EDA), and advanced SQL operations such as Common Table Expressions (CTEs) and Window Functions. The insights generated from this project help understand workforce performance, salary distribution, hiring trends, promotions, and employee productivity.

2. Project Objectives

- Analyze employee distribution across departments.
- Evaluate salary trends across the organization.
- Analyze employee performance and identify top performers.
- Study overtime and productivity patterns.
- Analyze employee satisfaction levels.
- Understand promotion patterns and career growth.
- Analyze hiring trends over time.
- Demonstrate advanced SQL concepts (CTEs and Window Functions).
- Build an interactive HR Analytics dashboard in Power BI.

3. Dataset Information

Dataset Name:	Employee Performance and Productivity Dataset
Source:	Kaggle
Database Name:	employee_db
Table Name:	employee
Total Records:	100,000
Total Columns:	20

4. Database Schema

The `employee` table contains employee demographic details, salary information, performance scores, promotions, overtime hours, work experience, education level, and employee satisfaction scores. This schema is used as the foundation for all SQL analysis performed in this project.

5. Data Inspection

Data inspection is the first step in any data analysis project. In this phase, the dataset is examined to understand its structure, total number of records, available columns, and overall data quality before performing further analysis.

Query 1 — Total Records

```
1 SELECT COUNT(*) AS Total_Employee FROM employee;
```

Query 2 — First 10 Rows

```
1 SELECT * FROM employee LIMIT 10;
```

Query 3 — Last 10 Rows

```
1 SELECT * FROM employee ORDER BY Employee_ID DESC;
```

Query 4 — Total Columns

```
1 SELECT COUNT(*) AS Total_Columns
2 FROM INFORMATION_SCHEMA.COLUMNS
3 WHERE TABLE_SCHEMA = 'employee_db'
4 AND TABLE_NAME = 'employee';
```

Query 5 — Table Structure

```
1 DESCRIBE employee;
```

6. Data Quality Check

The purpose of data quality checking is to verify that the dataset is complete, accurate, and consistent. This step includes checking for missing values, duplicate records, unique values, and validating important fields before starting exploratory data analysis.

Query 1 — Missing Values Check

```
1 SELECT
2     SUM(Employee_ID IS NULL) AS Employee_ID_Null,
3     SUM(Department IS NULL) AS Department_Null,
4     SUM(Gender IS NULL) AS Gender_Null,
5     SUM(Age IS NULL) AS Age_Null,
6     SUM(Job_Title IS NULL) AS Job_Title_Null,
7     SUM(Hire_Date IS NULL) AS Hire_Date_Null,
8     SUM(Performance_Score IS NULL) AS Performance_Null,
9     SUM(Monthly_Salary IS NULL) AS Salary_Null,
10    SUM(Employee_Satisfaction_Score IS NULL) AS Satisfaction_Null
11 FROM employee;
```

Query 2 — Duplicate Employees

```
1 SELECT Employee_ID, COUNT(*) AS Total
2 FROM employee
3 GROUP BY Employee_ID
4 HAVING COUNT(*) > 1;
```

Query 3 — Departments

```
1 SELECT DISTINCT Department FROM employee;
```

Query 4 — Job Titles

```
1 SELECT DISTINCT Job_Title FROM employee ORDER BY Job_Title;
```

Query 5 — Gender Values

```
1 SELECT Gender, COUNT(*) AS Total FROM employee GROUP BY Gender;
```

Query 6 — Resigned Values

```
1 SELECT Resigned, COUNT(*) AS Total FROM employee GROUP BY Resigned;
```

Query 7 — Salary Range

```
1 SELECT MIN(Monthly_Salary) AS Minimum_Salary,
2        MAX(Monthly_Salary) AS Maximum_Salary,
3        AVG(Monthly_Salary) AS Average_Salary
4 FROM employee;
```

Query 8 — Performance Score

```
1 SELECT MIN(Performance_Score) AS Minimum_Performance,
2        MAX(Performance_Score) AS Maximum_Performance
3 FROM employee;
```

7. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is performed to understand the characteristics of the employee dataset. The analysis focuses on employee distribution, salary, performance, overtime, satisfaction, promotions, experience, and hiring trends using SQL queries.

7.1 Section A — Employee Overview

This section provides an overview of the workforce by analyzing employee distribution across departments, job titles, gender, education level, and age.

Query 1 — Department-wise Employee Count

```
1 SELECT Department, COUNT(*) AS Total_Employees
2 FROM employee
3 GROUP BY Department
4 ORDER BY Total_Employees DESC;
```

Query 2 — Gender Distribution

```
1 SELECT Gender, COUNT(*) AS Total FROM employee GROUP BY Gender;
```

Query 3 — Job Title Distribution

```
1 SELECT Job_Title, COUNT(*) AS Total
2 FROM employee
3 GROUP BY Job_Title
4 ORDER BY Total DESC;
```

Query 4 — Education Level Distribution

```
1 SELECT Education_Level, COUNT(*) AS Total
2 FROM employee
3 GROUP BY Education_Level
4 ORDER BY Total DESC;
```

Query 5 — Average Employee Age

```
1 SELECT ROUND(AVG(Age),2) AS Average_Age FROM employee;
```

7.2 Section B — Salary Analysis

This section analyzes employee salary data across different departments, including average salary, salary distribution, departmental salary expenses, and salary categorization.

Query 6 — Department-wise Average Salary

```
1 SELECT Department, ROUND(AVG(Monthly_Salary),2) AS Average_Salary
2 FROM employee
3 GROUP BY Department
4 ORDER BY Average_Salary DESC;
```

Query 7 — Highest Salary in Each Department

```
1 SELECT Department, MAX(Monthly_Salary) AS Highest_Salary
2 FROM employee
3 GROUP BY Department
4 ORDER BY Highest_Salary DESC;
```

Query 8 — Lowest Salary in Each Department

```
1 SELECT Department, MIN(Monthly_Salary) AS Lowest_Salary
2 FROM employee
3 GROUP BY Department
4 ORDER BY Lowest_Salary DESC;
```

Query 9 — Total Salary Expense by Department

```
1 SELECT Department, SUM(Monthly_Salary) AS Total_Salary
2 FROM employee
3 GROUP BY Department
4 ORDER BY Total_Salary DESC;
```

Query 10 — Salary Category using CASE

```
1 SELECT Employee_ID, Department, Monthly_Salary,
2        CASE
3            WHEN Monthly_Salary < 5000 THEN 'Low Salary'
4            WHEN Monthly_Salary BETWEEN 5000 AND 8000 THEN 'Medium Salary'
5            ELSE 'High Salary'
6        END AS Salary_Category
7 FROM employee;
```

7.3 Section C — Performance Analysis

This section evaluates employee performance using performance scores, helping identify top-performing employees and compare departmental performance.

Query 11 — Average Performance by Department

```
1 SELECT Department, ROUND(AVG(Performance_Score),2) AS Avg_Performance
2 FROM employee
3 GROUP BY Department
4 ORDER BY Avg_Performance DESC;
```

Query 12 — Highest Performance Score

```
1 SELECT MAX(Performance_Score) AS Highest_Performance FROM employee;
```

Query 13 — Employees with Highest Performance

```
1 SELECT Employee_ID, Department, Performance_Score
2 FROM employee
3 WHERE Performance_Score = (
4     SELECT MAX(Performance_Score) FROM employee
5 );
```

7.4 Section D — Overtime & Productivity Analysis

This section examines employee overtime hours across departments and job roles to understand workload distribution and productivity patterns.

Query D1 — Department-wise Average Overtime

```
1 SELECT Department, ROUND(AVG(Overtime_Hours),2) AS Avg_Overtime
2 FROM employee
3 GROUP BY Department
4 ORDER BY Avg_Overtime DESC;
```

Query D2 — Top 10 Employees with Highest Overtime

```
1 SELECT Employee_ID, Department, Overtime_Hours
2 FROM employee
3 ORDER BY Overtime_Hours DESC
4 LIMIT 10;
```

Query D3 — Average Overtime by Job Title

```
1 SELECT Job_Title, ROUND(AVG(Overtime_Hours),2) AS Avg_Overtime
2 FROM employee
3 GROUP BY Job_Title
4 ORDER BY Avg_Overtime DESC;
```

Query D4 — Overtime Category (CASE)

```
1 SELECT Employee_ID, Overtime_Hours,
2     CASE
3         WHEN Overtime_Hours < 5 THEN 'Low'
4         WHEN Overtime_Hours BETWEEN 5 AND 15 THEN 'Medium'
5         ELSE 'High'
6     END AS Overtime_Category
7 FROM employee;
```

Query D5 — Performance vs Overtime

```
1 SELECT Performance_Score, ROUND(AVG(Overtime_Hours),2) AS
2     Avg_Overtime
3 FROM employee
4 GROUP BY Performance_Score
5 ORDER BY Performance_Score DESC;
```

7.5 Section E — Satisfaction Analysis

This section analyzes employee satisfaction scores across departments and job titles to understand employee engagement and workplace satisfaction.

Query E1 — Average Satisfaction by Department

```
1 SELECT Department, ROUND(AVG(Employee_Satisfaction_Score),2) AS
2     Avg_Satisfaction
3 FROM employee
4 GROUP BY Department
5 ORDER BY Avg_Satisfaction DESC;
```

Query E2 — Average Satisfaction by Job Title

```
1 SELECT Job_Title, ROUND(AVG(Employee_Satisfaction_Score),2) AS
2     Avg_Satisfaction
3 FROM employee
4 GROUP BY Job_Title
5 ORDER BY Avg_Satisfaction DESC;
```

Query E3 — Overall Satisfaction

```
1 SELECT ROUND(AVG(Employee_Satisfaction_Score),2) AS  
   Overall_Satisfaction  
2 FROM employee;
```

Query E4 — Satisfaction Level (CASE)

```
1 SELECT Employee_ID, Employee_Satisfaction_Score,  
2        CASE  
3            WHEN Employee_Satisfaction_Score < 2 THEN 'Low'  
4            WHEN Employee_Satisfaction_Score BETWEEN 2 AND 4 THEN 'Medium'  
5            ELSE 'High'  
6        END AS Satisfaction_Level  
7 FROM employee;
```

Query E5 — Satisfaction vs Performance

```
1 SELECT Performance_Score, ROUND(AVG(Employee_Satisfaction_Score),2)  
   AS Avg_Satisfaction  
2 FROM employee  
3 GROUP BY Performance_Score  
4 ORDER BY Performance_Score DESC;
```

7.6 Section F — Promotion Analysis

This section evaluates employee promotion patterns across departments and years of experience, helping identify career growth trends within the organization.

Query F1 — Promotion Distribution

```
1 SELECT Promotions, COUNT(*) AS Total_Employees  
2 FROM employee  
3 GROUP BY Promotions  
4 ORDER BY Promotions DESC;
```

Query F2 — Department-wise Total Promotions

```
1 SELECT Department, SUM(Promotions) AS Total_Promotions  
2 FROM employee  
3 GROUP BY Department  
4 ORDER BY Total_Promotions DESC;
```

Query F3 — Average Promotions by Department

```
1 SELECT Department, ROUND(AVG(Promotions),2) AS Avg_Promotions  
2 FROM employee  
3 GROUP BY Department  
4 ORDER BY Avg_Promotions DESC;
```

Query F4 — Promotion vs Experience

```
1 SELECT Years_At_Company , ROUND(AVG(Promotions),2) AS Avg_Promotion
2 FROM employee
3 GROUP BY Years_At_Company
4 ORDER BY Years_At_Company;
```

Query F5 — Promotion vs Performance

```
1 SELECT Performance_Score , ROUND(AVG(Promotions),2) AS Avg_Promotion
2 FROM employee
3 GROUP BY Performance_Score
4 ORDER BY Performance_Score DESC;
```

7.7 Section G — Experience Analysis

This section analyzes employee work experience and its relationship with salary and performance, providing insights into workforce maturity and career progression.

Query G1 — Average Experience

```
1 SELECT ROUND(AVG(Years_At_Company),2) AS Avg_Experience FROM employee
;
```

Query G2 — Experience Distribution

```
1 SELECT Years_At_Company , COUNT(*) AS Employees
2 FROM employee
3 GROUP BY Years_At_Company
4 ORDER BY Years_At_Company;
```

Query G3 — Experience vs Salary

```
1 SELECT Years_At_Company , ROUND(AVG(Monthly_Salary),2) AS Avg_Salary
2 FROM employee
3 GROUP BY Years_At_Company
4 ORDER BY Years_At_Company;
```

Query G4 — Experience vs Performance

```
1 SELECT Years_At_Company , ROUND(AVG(Performance_Score),2) AS
   Avg_Performance
2 FROM employee
3 GROUP BY Years_At_Company
4 ORDER BY Years_At_Company;
```

Query G5 — Top 10 Most Experienced Employees

```
1 SELECT * FROM employee ORDER BY Years_At_Company DESC LIMIT 10;
```

7.8 Section H — Hiring Trend (Date Functions)

This section analyzes hiring trends over different years and months using SQL date functions to understand recruitment patterns across time.

Query H1 — Hiring by Year

```
1 SELECT YEAR(Hire_Date) AS Hire_Year, COUNT(*) AS Total_Hires
2 FROM employee
3 GROUP BY YEAR(Hire_Date)
4 ORDER BY Hire_Year;
```

Query H2 — Hiring by Month

```
1 SELECT MONTH(Hire_Date) AS Hire_Month, COUNT(*) AS Total_Hires
2 FROM employee
3 GROUP BY MONTH(Hire_Date)
4 ORDER BY Hire_Month;
```

Query H3 — Average Salary by Hiring Year

```
1 SELECT YEAR(Hire_Date) AS Hire_Year, ROUND(AVG(Monthly_Salary),2) AS
   Avg_Salary
2 FROM employee
3 GROUP BY YEAR(Hire_Date)
4 ORDER BY Hire_Year;
```

Query H4 — Average Performance by Hiring Year

```
1 SELECT YEAR(Hire_Date) AS Hire_Year, ROUND(AVG(Performance_Score),2)
   AS Avg_Performance
2 FROM employee
3 GROUP BY YEAR(Hire_Date)
4 ORDER BY Hire_Year;
```

Query H5 — Employee Experience Using Hire Date

```
1 SELECT Employee_ID, Hire_Date,
2        TIMESTAMPDIFF(YEAR, Hire_Date, CURDATE()) AS Experience
3 FROM employee;
```

8. Advanced SQL

This section demonstrates advanced SQL techniques, including Common Table Expressions (CTEs) and Window Functions such as ROW_NUMBER(), RANK(), DENSE_RANK(), LEAD(), LAG(), and NTILE(). These concepts are widely used in real-world business analysis and reporting.

8.1 Section I — CTE (Common Table Expression)

I1 — Department-wise Average Salary using CTE

```
1 WITH AvgSalary AS (  
2     SELECT Department, ROUND(AVG(Monthly_Salary),2) AS Avg_Salary  
3     FROM employee  
4     GROUP BY Department  
5 )  
6 SELECT * FROM AvgSalary ORDER BY Avg_Salary DESC;
```

I2 — Departments with Average Salary Greater than 6400

```
1 WITH AvgSalary AS (  
2     SELECT Department, AVG(Monthly_Salary) AS Avg_Salary  
3     FROM employee  
4     GROUP BY Department  
5 )  
6 SELECT * FROM AvgSalary WHERE Avg_Salary > 6400;
```

I3 — Department-wise Average Performance using CTE

```
1 WITH AvgPerformance AS (  
2     SELECT Department, ROUND(AVG(Performance_Score),2) AS  
3         Avg_Performance  
4     FROM employee  
5     GROUP BY Department  
6 )  
7 SELECT * FROM AvgPerformance ORDER BY Avg_Performance DESC;
```

8.2 Section J — Window Functions

This section demonstrates advanced SQL window functions used for ranking, comparison, cumulative calculations, and analytical reporting.

J1 — ROW_NUMBER()

```
1 SELECT Employee_ID, Department, Monthly_Salary,  
2     ROW_NUMBER() OVER (  
3         PARTITION BY Department  
4         ORDER BY Monthly_Salary DESC  
5     ) AS Row_Num  
6 FROM employee;
```

J2 — RANK()

```
1 SELECT Employee_ID, Department, Monthly_Salary,  
2     RANK() OVER (  
3         PARTITION BY Department ORDER BY Monthly_Salary DESC  
4     ) AS Salary_Rank  
5 FROM employee;
```

J3 — DENSE_RANK()

```
1 SELECT Employee_ID, Department, Monthly_Salary,  
2     DENSE_RANK() OVER (  
3         PARTITION BY Department ORDER BY Monthly_Salary DESC
```

```
4 ) AS Salary_Dense_Rank
5 FROM employee;
```

J4 — Top 3 Highest Paid Employees from Every Department

```
1 WITH SalaryRank AS (
2     SELECT Employee_ID, Department, Monthly_Salary,
3         DENSE_RANK() OVER (
4             PARTITION BY Department ORDER BY Monthly_Salary DESC
5         ) AS Rank_No
6     FROM employee
7 )
8 SELECT * FROM SalaryRank WHERE Rank_No <= 3;
```

J5 — Lowest Paid Employee from Every Department

```
1 WITH SalaryRank AS (
2     SELECT Employee_ID, Department, Monthly_Salary,
3         ROW_NUMBER() OVER (
4             PARTITION BY Department ORDER BY Monthly_Salary
5         ) AS rn
6     FROM employee
7 )
8 SELECT * FROM SalaryRank WHERE rn = 1;
```

J6 — NTILE(): Divide Employees into 4 Salary Groups

```
1 SELECT Employee_ID, Department, Monthly_Salary,
2     NTILE(4) OVER (ORDER BY Monthly_Salary DESC) AS Salary_Quartile
3 FROM employee;
```

J7 — LEAD()

```
1 SELECT Employee_ID, Department, Monthly_Salary,
2     LEAD(Monthly_Salary) OVER (ORDER BY Monthly_Salary DESC) AS
3     Next_Salary
4 FROM employee;
```

J8 — LAG()

```
1 SELECT Employee_ID, Department, Monthly_Salary,
2     LAG(Monthly_Salary) OVER (ORDER BY Monthly_Salary DESC) AS
3     Previous_Salary
4 FROM employee;
```

J9 — FIRST_VALUE()

```
1 SELECT Employee_ID, Department, Monthly_Salary,
2     FIRST_VALUE(Monthly_Salary) OVER (
3         PARTITION BY Department ORDER BY Monthly_Salary DESC
4     ) AS Highest_Salary_Department
5 FROM employee;
```

J10 — LAST_VALUE()

```
1 SELECT Employee_ID, Department, Monthly_Salary,
2     LAST_VALUE(Monthly_Salary) OVER (
3         PARTITION BY Department ORDER BY Monthly_Salary DESC
4         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
5     ) AS Lowest_Salary_Department
6 FROM employee;
```

J11 — Highest Paid Employee in Each Department

```
1 WITH HighestSalary AS (
2     SELECT Employee_ID, Department, Monthly_Salary,
3         ROW_NUMBER() OVER (
4             PARTITION BY Department ORDER BY Monthly_Salary DESC
5         ) AS rn
6     FROM employee
7 )
8 SELECT * FROM HighestSalary WHERE rn = 1;
```

J12 — Lowest Paid Employee in Each Department

```
1 WITH LowestSalary AS (
2     SELECT Employee_ID, Department, Monthly_Salary,
3         ROW_NUMBER() OVER (
4             PARTITION BY Department ORDER BY Monthly_Salary
5         ) AS rn
6     FROM employee
7 )
8 SELECT * FROM LowestSalary WHERE rn = 1;
```

J13 — Top 5 Highest Performing Employees

```
1 SELECT Employee_ID, Department, Performance_Score,
2     DENSE_RANK() OVER (ORDER BY Performance_Score DESC) AS
3     Performance_Rank
4 FROM employee;
```

J14 — Running Total Salary

```
1 SELECT Employee_ID, Monthly_Salary,
2     SUM(Monthly_Salary) OVER (ORDER BY Employee_ID) AS
3     Running_Total_Salary
4 FROM employee;
```

J15 — Average Salary by Department (Window Function)

```
1 SELECT Employee_ID, Department, Monthly_Salary,
2     AVG(Monthly_Salary) OVER (
3         PARTITION BY Department
4     ) AS Department_Avg_Salary
5 FROM employee;
```

9. Power BI Dashboard

An interactive HR Analytics dashboard was built in Microsoft Power BI to visualize the insights generated from the SQL analysis. The dashboard presents key metrics through interactive charts and filters, including:

- Employee distribution by department, gender, and education level.
- Average and total salary expenses across departments.
- Performance and satisfaction score comparisons.
- Overtime and productivity patterns.
- Hiring trends across years and months.
- Promotion and career growth insights.



Employee Performance and Productivity Dashboard developed in Microsoft Power BI.

10. Business

Insights

Based on the SQL analysis and Power BI visualizations, the following key business insights were derived:

- Employee distribution varies across departments, highlighting where the organization concentrates its workforce.
- Salary trends reveal departmental pay differences and help identify high-cost and high-value areas.
- Performance scores correlate with experience and promotions, indicating clear career progression patterns.
- Overtime analysis shows workload distribution and helps identify potential burnout risks.
- Satisfaction scores provide insight into employee engagement across departments and roles.
- Hiring trends reveal seasonal and yearly recruitment patterns useful for workforce planning.

11. Conclusion

This project successfully demonstrated the use of SQL and Power BI to analyze a large dataset of 100,000 employee records. Through data inspection, quality checks, exploratory data analysis, and advanced SQL techniques such as CTEs and window functions, meaningful insights were extracted regarding workforce composition, salary distribution, performance, satisfaction, promotions, and hiring trends.

The combination of structured SQL querying and interactive Power BI visualization provides a powerful approach to HR analytics, enabling data-driven decision-making for workforce management and organizational planning.

End of Report

Prepared by Roshan Kumar